MAKİNE ÖĞRENMESİNE GİRİŞ VE MODEL DEĞERLENDİRME

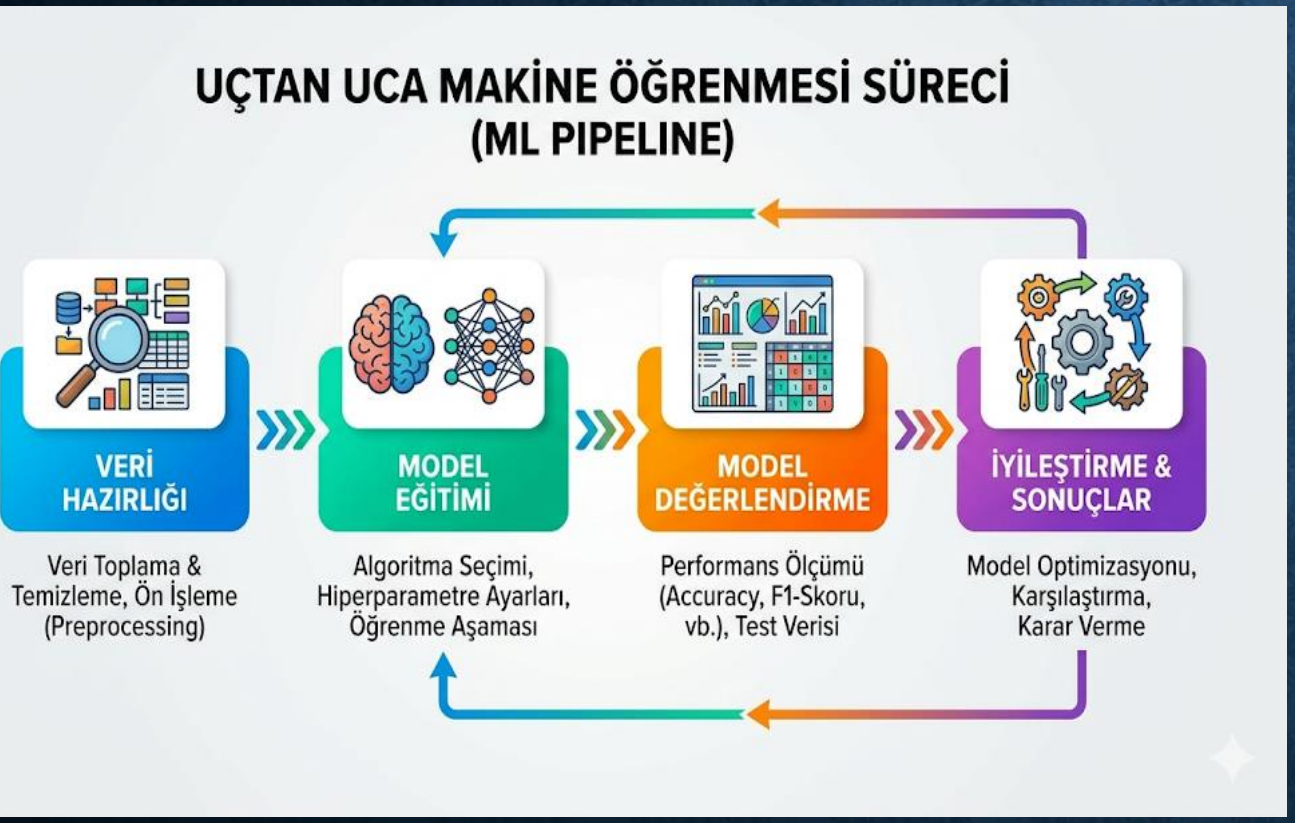
Hazırlayan: Hamdi Furkan YILMAZ

Öğrenci No: 2322705007

NELER KONUŞACAĞIZ ?

- ML Pipeline (Uçtan Uca Süreç)
- Veri Bölme Stratejileri
- Bias ve Variance Dengesi
- Değerlendirme Metrikleri
- Mini Proje Çıktılarımız

ML PİPELİNE (END-TO-END SÜREÇ)



- **Veri:** Toplama, temizleme ve hazırlık.
- **Model:** Eğitme ve algoritmaya örüntüleri öğretme.
- **Değerlendirme:** Test etme ve performansı ölçme.
- **İyileştirme:** Hataları giderip süreci başa sarma.

VERİ BÖLME STRATEJİLERİ

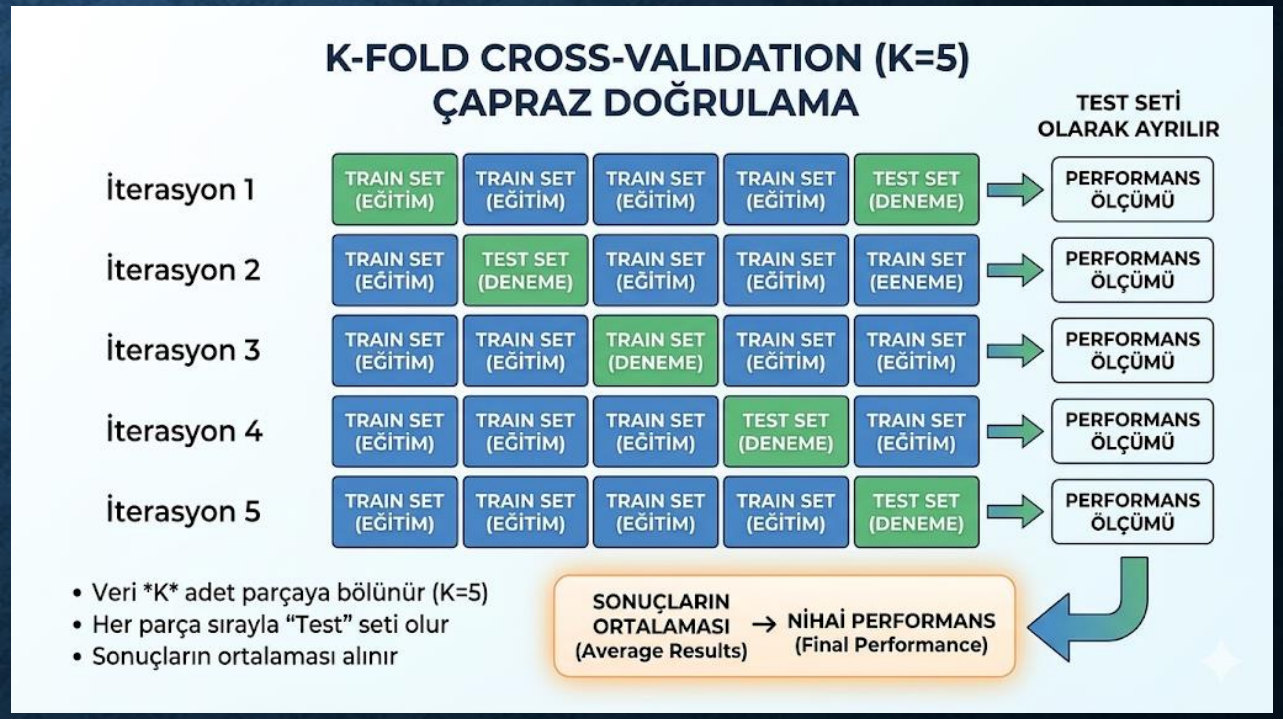
- **Eğitim (Train) Seti (%70):** Modelin verideki örüntüleri ve kuralları öğrendiği ana aşama.

- **Geliştirme (Validation/Dev) Seti (%15):** Modelin performansını test edip parametre ayarlarını yaptığımız kontrol aşaması.

- **Test Seti (%15):** Modelin nihai performansını ölçtüğümüz, **daha önce hiç görmediği** verilerden oluşan final aşaması.

K-FOLD CROSS VALİDATİON (ÇAPRAZ DOĞRULAMA)

- Veri az olduğunda kurtarıcıdır.
- Veri seti *K* adet parçaya bölünür.
- Her parça sırayla "Test" seti olur.
- Sonuçların ortalaması alınır.



PROJEMİZDE VERİ BÖLME İŞLEMİ

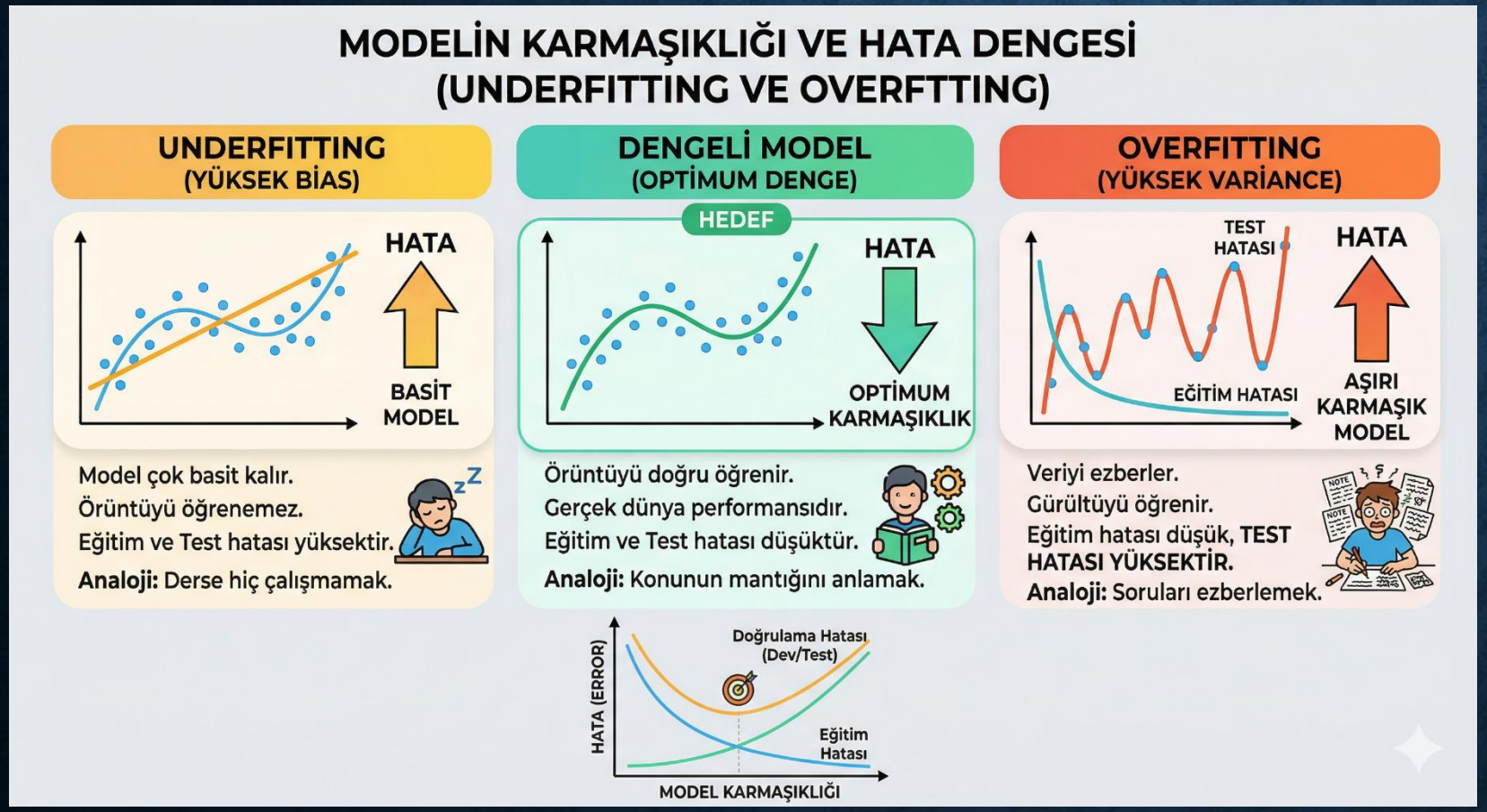
```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Önce veriyi %70 Eğitim (Train) ve %30 Geçici (Temp) olarak ikiye bölüyoruz.
# random_state=42 kullanıyoruz ki kodu her çalıştırdığımızda aynı rastgele dağılımı versin (raporlama için önemli).
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)

# Şimdi o %30'luk Geçici (Temp) veriyi tam ortadan ikiye bölüyoruz: %15 Dev (Validation) ve %15 Test.
X_dev, X_test, y_dev, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

# Yapay Sinir Ağları (TensorFlow) için veriyi standartlaştırıyoruz (Ölçeklendirme)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_dev_scaled = scaler.transform(X_dev)
X_test_scaled = scaler.transform(X_test)

print("Veri seti başarıyla 3 parçaya bölündü ve ölçeklendirildi:")
print(f"Eğitim (Train) Seti: {X_train.shape[0]} satır")
print(f"Geliştirme (Dev/Validation) Seti: {X_dev.shape[0]} satır")
print(f"Test Seti: {X_test.shape[0]} satır")
```

```
Veri seti başarıyla 3 parçaya bölündü ve ölçeklendirildi:
Eğitim (Train) Seti: 398 satır
Geliştirme (Dev/Validation) Seti: 85 satır
Test Seti: 86 satır
```

- **Toplam Veri Sayısı:** 569 hasta (örnek)
- **Eğitim (Train) Seti:** 398 adet
- **Geliştirme (Dev) Seti:** 85 adet
- **Test Seti:** 86 adet

- **Train Oranı:** 398 / 569 = **%69.95**
  (Hedefimiz %70'ti)
- **Dev Oranı:** 85 / 569 = **%14.94**
  (Hedefimiz %15'ti)
- **Test Oranı:** 86 / 569 = **%15.11**
  (Hedefimiz %15'ti)

BIAS VE VARIANCE (MODELIN EZBERLEMESI)



- **Underfitting (Yüksek Bias):** Modelin çok basit kalması ve konuyu hiç öğrenememesi.

- **Overfitting (Yüksek Variance):** Modelin elindeki veriyi ezberlemesi ancak daha önce görmediği yeni verilerde çuvallaması.

- **Hedef (Trade-off):** Aşırı basitlik ile ezbercilik arasındaki o mükemmel dengeyi bulmak.

MODEL BAŞARISINI NASIL ÖLÇÜYORUZ ?

- **Temel Matris:** TP (Gerçek Pozitif), TN (Gerçek Negatif), FP (Yanlış Pozitif), FN (Yanlış Negatif).

- **Accuracy (Doğruluk):** Doğru tahminlerin (TP+TN) toplam örneklere oranı.

- **Precision (Kesinlik):** Pozitif dediklerimizin gerçekten pozitif olma oranı.

- **Recall (Duyarlılık):** Gerçek pozitifleri kaçırmadan yakalama başarımız.

- **F1-Skoru:** Kesinlik ve Duyarlılığın denge noktası (Harmonik ortalaması).

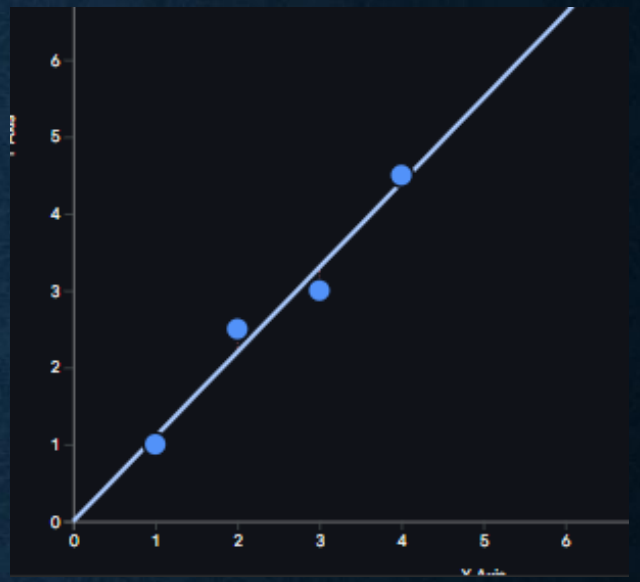
REGRESYON METRİKLERİ (SAYISAL TAHMİN)

•**Regresyon:** Kategori değil, **sayı** tahmin eder (Örn: Ev fiyatı).

•Hedef "%100 bilmek" değil, **"en az sapmayı"** yapmaktır.

•**MAE (Ortalama Hata):** Tahminlerimiz ortalama ne kadar saptı?
(Örn: "Ev fiyatlarını ortalama 20 bin TL eksik/fazla tahmin ediyoruz.")

•**MSE (Karesel Hata):** Büyük hataları cezalandırma sistemi.
(Amacı: Modelin uçuk kaçık hatalar yapmasını engellemek.)




```python
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Diyelim ki gerçek ev fiyatları ve modelimizin tahminleri şöyle (Milyon TL):
gercek_fiyatlar = [3.0, 4.5, 2.0, 5.5]
tahminler       = [2.8, 4.5, 2.5, 5.0]

# 1. MAE (Ortalama Sapma)
mae = mean_absolute_error(gercek_fiyatlar, tahminler)
print(f"Ortalama Hata (MAE): {mae:.2f} Milyon TL")

# 2. MSE (Karesel Hata)
mse = mean_squared_error(gercek_fiyatlar, tahminler)
print(f"Karesel Hata (MSE): {mse:.3f}")
```

```
Ortalama Hata (MAE): 0.30 Milyon TL
Karesel Hata (MSE): 0.135
```

TENSORFLOW MODELİMİZİN BAŞARISI

```python
import tensorflow as tf
from sklearn.metrics import accuracy_score
import numpy as np

# 1. Modeli tanımlama (Derin Öğrenme Modeli)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid') # İkili sınıflandırma (0 veya 1) için sigmoid kullanılır
])

# Modeli derleme
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 2. Modeli SADECE Eğitim (Train) verisi ile eğitiyoruz
# verbose=0 ile ekrandaki kalabalık eğitim adımlarını gizliyoruz
model.fit(X_train_scaled, y_train, epochs=50, batch_size=16, verbose=0)

# 3. Eğitilmiş modele 3 farklı veri seti için tahminler yaptırıyoruz
# TensorFlow model.predict, 0 ile 1 arasında olasılıklar döndürür.
# Bunları 0.5 eşik değeri ile 0 veya 1 sınıflarına (Kötü Huylu/İyi Huylu) çeviriyoruz.
train_tahmin = (model.predict(X_train_scaled, verbose=0) > 0.5).astype(int).flatten()
dev_tahmin = (model.predict(X_dev_scaled, verbose=0) > 0.5).astype(int).flatten()
test_tahmin = (model.predict(X_test_scaled, verbose=0) > 0.5).astype(int).flatten()
```

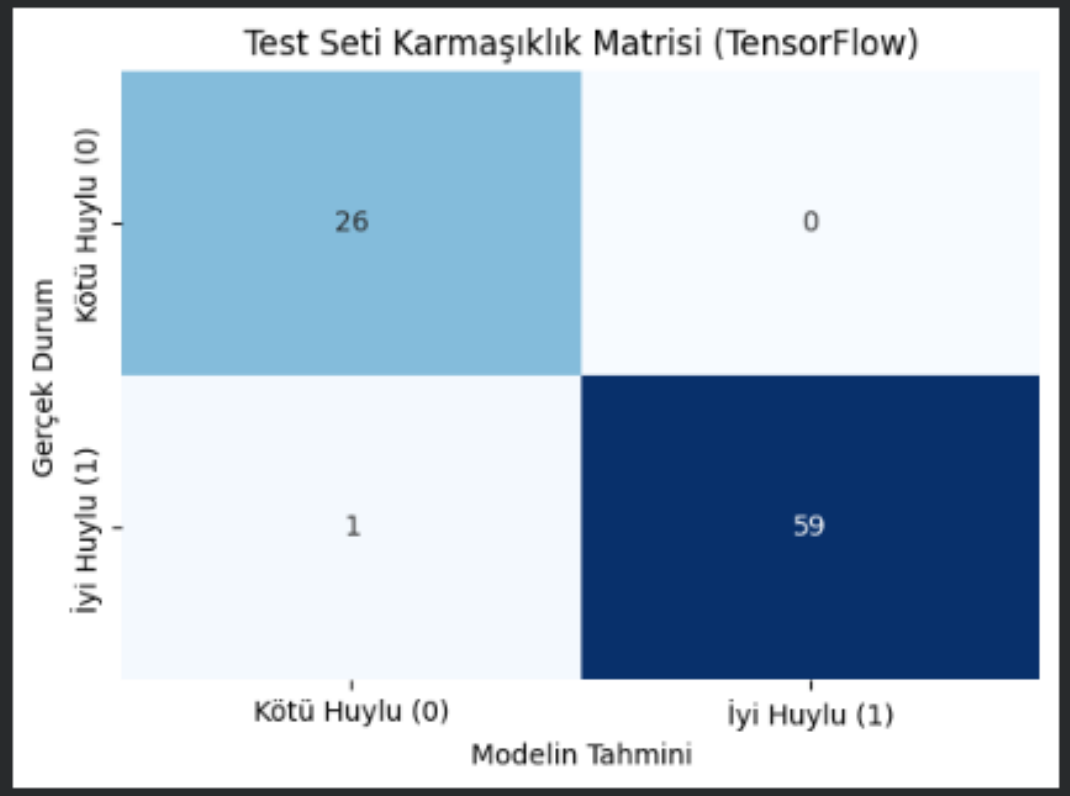
TENSORFLOW MODELİMİZİN BAŞARISI

```python
# 4. Başarı oranlarını (Accuracy) hesaplıyoruz
train_basari = accuracy_score(y_train, train_tahmin)
dev_basari = accuracy_score(y_dev, dev_tahmin)
test_basari = accuracy_score(y_test, test_tahmin)

# Sonuçları Yazdırma
print("--- Model Başarı Oranları (Accuracy) ---")
print(f"Eğitim (Train) Seti Başarısı: %{train_basari*100:.2f}  <-- (Modelin bildiği veri, ezber olabilir)")
print(f"Geliştirme (Dev) Seti Başarısı: %{dev_basari*100:.2f}  <-- (Ayar yaptığımız, kontrol verisi)")
print(f"Test Seti Başarısı:       %{test_basari*100:.2f}  <-- (Gerçek dünya performansı, İLK DEFA gördüğü veri)")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
--- Model Başarı Oranları (Accuracy) ---
Eğitim (Train) Seti Başarısı: %99.25  <-- (Modelin bildiği veri, ezber olabilir)
Geliştirme (Dev) Seti Başarısı: %96.47  <-- (Ayar yaptığımız, kontrol verisi)
Test Seti Başarısı:       %98.84  <-- (Gerçek dünya performansı, İLK DEFA gördüğü veri)
```

- **Sonuç - 1:** Modelimiz Eğitim setinde örüntüleri neredeyse tam olarak kavradı (%99,25).
- **Sonuç - 2:** Daha önce hiç görmediği Test setinde %96,47 başarı gösterdi.
- **Değerlendirme:** Modelimiz ezber (overfitting) yapmamış, gerçekten öğrenmiştir !

TEST SETİ KARMAŞIKLIK MATRİSİ (CONFUSİON MATRİX)




```
--- Detaylı Sınıflandırma Raporu (Test Seti) ---
                precision    recall  f1-score   support

Kötü Huylu (0)       0.96      1.00      0.98        26
 İyi Huylu (1)       1.00      0.98      0.99        60

      accuracy                           0.99        86
     macro avg       0.98      0.99      0.99        86
  weighted avg       0.99      0.99      0.99        86
```

- 86 test verisinin 85'ü doğru, sadece 1'ini yanlış tahmin edilmiştir.
  - **Genel Başarı (Accuracy):** %99
  - **Kötü Huylu Tümör Tespiti:** 26'da 26 Kusursuz Başarı (Recall: %100)
  - **Tek Hata (Yanlış Alarm):** 1 iyi huylu hastaya "şüpheli" uyarısı.
  - **Tıbbi Sonuç:** Gözden kaçan, tehlikede olan HİÇBİR hasta yok!

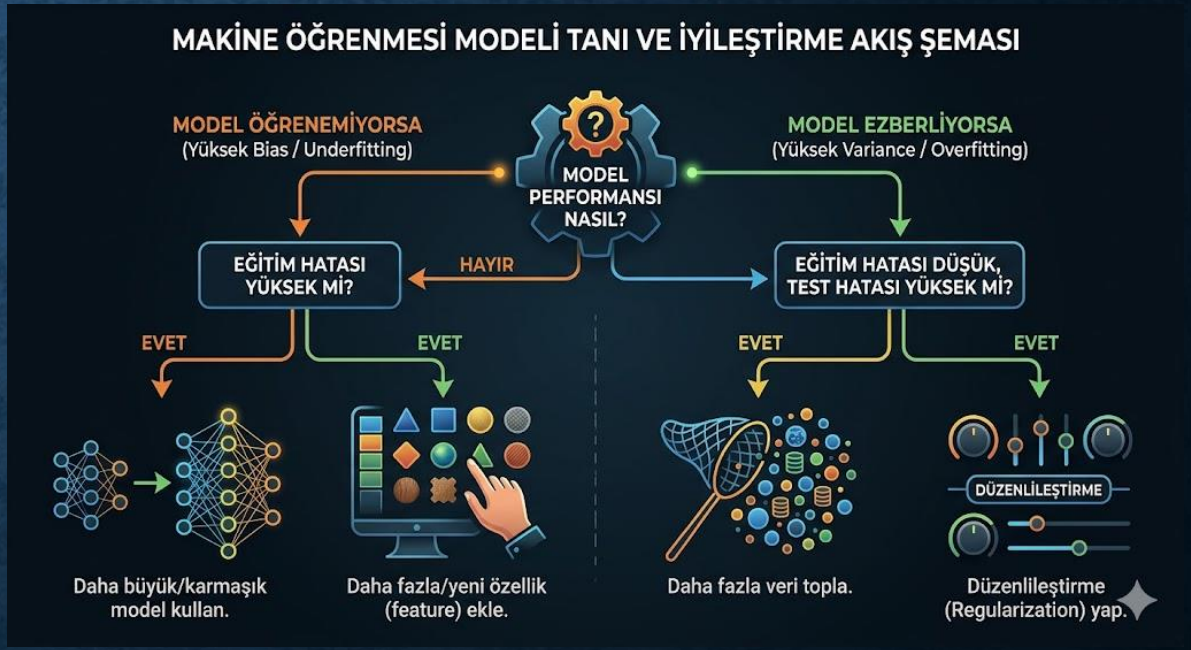
TEMEL ML REÇETESİ (SORUN ÇÖZME REHBERİ)

Model Öğrenemiyorsa (Yüksek Bias / Underfitting):

•Daha büyük/karmaşık model kullan.
•Daha fazla/yeni özellik ekle.

Model Ezberliyorsa (Yüksek Variance / Overfitting):

•Daha fazla veri topla.
•Düzenlileştirme yap.



TEŞEKKÜRLER